

Weather Explorer

A small full-stack weather explorer built for the InRisk Labs case study. It fetches historical daily weather from the [Open-Meteo](#) archive API, stores the raw JSON in object storage, and serves a dashboard to trigger/store, browse, and visualize those files.

Live demo & repo - Live demo (frontend): <https://frontend-one-roan-33.vercel.app> - Backend API: <https://weather-explorer-api.vercel.app> (auto-docs at [/docs](#)) - GitHub: <https://github.com/archanamalluri5-eng/weather-explorer> - Last verified live: **2026-08-08**

Repository layout

```
weather-explorer/
├── backend/                # FastAPI service
│   ├── app/
│   │   ├── app_factory.py  # App factory, CORS, error handling
│   │   ├── config.py       # Env-driven settings
│   │   ├── validation.py   # Shared validation helpers
│   │   ├── routes/         # HTTP endpoints
│   │   ├── services/       # Open-Meteo client + store orchestrator
│   │   └── storage/        # Local / GCS / S3 backends behind one
├── interface
│   ├── api/index.py        # Vercel (ASGI) entrypoint
│   ├── vercel.json         # Vercel function config
│   ├── tests/              # pytest suite (29 tests)
│   ├── requirements.txt
│   └── Dockerfile          # Cloud Run / Render image
└── frontend/              # React + Vite + Tailwind dashboard
    └── src/
        ├── api.js          # fetch wrapper
        └── weather.js      # payload -> rows / summary helpers
```

```
└─ components/      # InputPanel, FileList, TemperatureChart,
WeatherTable
```

API

All endpoints are under `/api`. Interactive docs at `/docs` (Swagger).

POST `/api/store-weather-data`

```
{ "latitude": 51.5074, "longitude": -0.1278,
  "start_date": "2024-01-01", "end_date": "2024-01-10" }
```

- Validates coordinates, dates, and the ≤ 31 -day range.
- Calls Open-Meteo daily-history with `temperature_2m_max`, `temperature_2m_min`, `apparent_temperature_max`, `apparent_temperature_min`.
- Stores the **full raw API JSON** as `weather_<lat>_<lon>_<start>_<end>_<timestamp>.json` in the bucket.
- Returns `{"status": "ok", "file": "<name>"}`. Errors $\rightarrow$ `400 / 502` with `{"status": "error", "message": "..."}.`

GET `/api/list-weather-files`

```
{ "files": [ { "name": "...", "size": 763, "created_at": "2026-08-
08T12:50:47Z" } ] }
```

GET `/api/weather-file-content/{file}`

Returns the stored JSON. Missing/invalid $\rightarrow$ `404` with `{"status": "error", "message": "not found"}`.

Backend design decisions

- **Storage is behind an interface.** `StorageBackend` (`put/get/list/exists`) has three implementations: local filesystem (`dev/tests`), Google Cloud Storage, and AWS S3. `get_storage()` picks one from `env ( STORAGE_BACKEND )` or auto-detects (GCP project → GCS, AWS creds → S3, otherwise local). This means the app and tests never depend on a real cloud, and switching buckets is a config change.
- **Cloud SDKs are lazy-imported.** `google-cloud-storage` / `boto3` only load when that backend is used, so the free-tier-friendly local path has zero heavy deps.
- **Efficient listing.** Both cloud backends use the SDK's native listing/pagination (`list_blobs` generator, `list_objects_v2` paginator) instead of brute-force scans.
- **Validation is centralized** in `validation.py` and reused by routes and the service layer, returning 400 with a structured body.
- **Structured errors.** FastAPI normally wraps `HTTPException.detail` in `{"detail": ...}`; a global handler unwraps it so every error body is exactly `{"status": "error", "message": "..."} per the spec.`
- **Open-Meteo client** verifies the response is valid JSON before storing, retries transient failures, and maps upstream errors to `502`.
- **Naming.** Coordinates are rounded to 4 decimals in filenames, which keeps names predictable and sortable while matching the request semantics.
- **CORS** is configurable via `CORS_ORIGINS` (`*` for dev, your frontend domain in production).

Frontend design decisions

- **Vite + React + Tailwind CSS v4 + Recharts** for the line chart.
 - **Works off stored files** — after a fetch, the app re-lists files and the dashboard only makes API calls when the user clicks a file. No repeated upstream Open-Meteo calls (the API stores, the UI browses).
 - **Pagination** with 10/20/50 rows-per-page, client-side over the daily rows.
 - **Responsive** grid: input + file list stack above the visualization on mobile/tablet, side-by-side on desktop.
 - **Loading/error states** everywhere: per-action spinners, inline form validation, and a dismissible global error banner.
 - The Vite dev server proxies `/api` to `http://localhost:8000`, so no CORS config is needed locally. In production, `VITE_API_BASE_URL` points at the deployed backend.
-

Running locally

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate          # Windows; on macOS/Linux: source
.venv/bin/activate
pip install -r requirements-dev.txt
python run.py                   # http://localhost:8000 (docs at /docs)
```

Frontend

```
cd frontend
npm install
npm run dev                     # http://localhost:5173 (proxies /api to
backend)
```

Tests

```
cd backend
.venv\Scripts\python -m pytest
```

Using real cloud storage

Free-tier resources only (no credit card / paid plan required):

Provider	Setup	Env vars
GCS	<code>gcloud auth application-default login</code> or set <code>GOOGLE_APPLICATION_CREDENTIALS</code> ; create a bucket	<code>STORAGE_BACKEND=gcs</code> , <code>GOOGLE_CLOUD_PROJECT=...</code> , <code>STORAGE_BUCKET=...</code>

Provider	Setup	Env vars
S3	<code>aws configure</code> (or IAM role); create a bucket	<code>STORAGE_BACKEND=s3</code> , <code>AWS_REGION=...</code> , <code>STORAGE_BUCKET=...</code>

AWS free tier includes 5 GB of S3 storage (plenty). GCS has a free tier for storage with `gcloud storage buckets create gs://...`.

Deployment

Both pieces currently run on Vercel (free tier):

- **Frontend** (`frontend/`): Vercel project `frontend` , build `npm run build` , output `dist` , env `VITE_API_BASE_URL=https://weather-explorer-api.vercel.app/api` .
- **Backend** (`backend/`): Vercel project `weather-explorer-api` , Python/ASGI runtime, entrypoint `api/index.py` (see `backend/vercel.json`). Env: `STORAGE_BACKEND=local` , `LOCAL_DATA_DIR=/tmp/weather-data` .

To redeploy after a change:

```
cd backend && vercel --prod && cd ../frontend && vercel --prod
```

New deploys get fresh stable aliases (`weather-explorer-api.vercel.app` and `frontend-one-roan-33.vercel.app`); if Vercel changes an alias, update this README and the frontend's `VITE_API_BASE_URL` .

Alternative: Render (blueprint included)

The repo still ships `render.yaml` (one-click "New + Blueprint" on Render, free web service) and `backend/Dockerfile` (Cloud Run). Those paths are not currently used for the live demo but are kept as documented alternatives.

Storage on the live demo

The live backend uses the **local** storage backend writing to `/tmp` . On Vercel serverless that directory is **ephemeral** — files persist across requests while the function instance is

warm but may be lost on a cold start. That is acceptable for the demo; for durable storage point `STORAGE_BACKEND` at GCS or S3 (free tiers) using the same env-switchable interface (see below).

Last verified live

2026-08-08 — `POST /store-weather-data`, `GET /list-weather-files`, and `GET /weather-file-content/{file}` all verified against the live URLs above, including a real Open-Meteo fetch (New York, June 2024) and CORS from the frontend origin.

Assumptions

- "Full API JSON" is interpreted as the exact response body Open-Meteo returns for the requested variables — stored byte-for-byte so the raw payload can be re-processed later.
- `timezone=auto` is requested from Open-Meteo so daily buckets line up with the location's local days rather than UTC.
- The 31-day cap (inclusive, so max 31 calendar days) matches the spec and also keeps chart/table interaction snappy.
- Local storage backend is the zero-cost default so the repo is fully runnable without any cloud account; production uses GCS or S3 behind the same interface.

Libraries used

- **Backend:** FastAPI, Uvicorn, Pydantic, httpx, google-cloud-storage, boto3, pytest.
- **Frontend:** React 18, Vite 6, Tailwind CSS v4, Recharts v3.